

Fundamentals

Codebase Orientation 🎧

How to quickly understand an unfamiliar multi-file project before you start prompting.

Most AI-enabled coding interviews drop you into a multi-file project with existing classes, data models, and logic already in place. Understanding what you're looking at before you touch anything is the single most important thing you can do in the first few minutes.

If you're in an open-ended interview starting from scratch, most of this doesn't apply, but the underlying skill of reading unfamiliar code quickly is worth developing regardless.

Why orientation comes first

Candidates who skip orientation and start prompting immediately make worse decisions for the rest of the interview. They ask the AI to implement something that conflicts with an existing pattern in the codebase. They miss a helper class that already does what they're trying to build. They waste time debugging because they didn't understand the data model.

Interviewers notice this. When you jump straight to prompting without reading, **it signals that you'd do the same thing on a real team: start changing code you don't understand.** That's not the impression you want to leave.

Understanding the project structure

Start with the big picture and work your way in. Build a mental map of how the pieces fit together, focusing on structure and relationships rather than line-by-line details.



At a real company, a colleague helping you get up to speed will often give you "code pointers": specific files or functions to read that unlock how the whole system fits together. In an interview, nobody gives you those. Your job is to find them yourself. There are usually just a few key spots in any codebase that, once you understand them, make everything else click.

1. **Entry points** - The first thing to find is the entry point. Where does execution start? If it's a web app, where do requests come in? If it's a CLI tool, where's the main function? Once you've found that, you can trace the flow of data through the system, and the rest of the codebase starts to organize itself in your head. You'll see which classes get instantiated, which methods get called, and how the different pieces hand off work to each other.
2. **Key functions** - Most codebases have a hard core: the complex function or method everything else is built around. Understanding how it's called, what it takes as input, and what it returns often tells you more about the system than reading a dozen simpler files. Look for the function that does the real work.
3. **Class hierarchy and data models** - From the entry point, the next thing to map out is the class hierarchy and the data models. What are the core objects and how do they relate to each other? If there's a `Game` class that holds a `Board` which contains `Cell` objects, understanding that hierarchy tells you a lot about how the system works before you read a single method body. Spend time with the data models specifically, because almost every bug you'll encounter later traces back to a misunderstanding of what the data looks like and how it moves through the system.
4. **Architectural patterns** - Before reading individual files, run `find . -type f` or just browse the directory tree. A flat list of filenames tells you a surprising amount about the architecture — whether this is MVC, layered, modular, etc. From there, pay attention to whether there's clear separation of concerns. If the codebase uses a repository pattern for data access, you know not to put database calls directly in the controller, even if the AI suggests it. Working with the existing patterns rather than against them is one of the clearest signals of a strong candidate.
5. **Public interfaces and encapsulation** - Scan for what's public vs. private on the key classes. If a `Deck` has a `getCards()` method, the AI needs to know to use it — not reach directly into the internal `cards` array. The AI only sees the file you're currently in; knowing the interfaces across the codebase is your job.
6. **State management** - Look for state management patterns. Where is state stored and how does it change? Is there a single source of truth or is state spread across multiple objects? Understanding this tells you where side effects happen and where bugs are most likely to hide.
7. **Existing tests** - If there are test files, read them. Tests are often the clearest documentation of what the code is supposed to do, and they give you a way to verify your

understanding. Run them. If they all pass, you know the existing code works as expected. If any fail, that might be part of the problem you're supposed to solve.



If the environment supports it, get a debugger running early. Being able to set a breakpoint and step through the code is faster than reading it cold, and it instantly shows you the real data shapes as they flow through the system.

8. **Constraints and assumptions** - Scan for constraints and assumptions. Comments or config files sometimes hint at performance requirements or known limitations. If a comment says "this works for inputs up to 10,000" and one of the later phases asks you to handle 1 million, you've just gotten a preview of what's coming.

Using AI to help you orient

AI is genuinely useful during the orientation phase, but the way you use it matters. The goal is to accelerate your understanding, not to outsource it.

One of the best moves is to ask the AI to add detailed comments to each class and method in the codebase. This keeps you in the code rather than bouncing back and forth to a chat window. You can read through the files naturally, with the AI's explanations right there inline, and build your mental model as you go. It's faster than asking one question at a time in chat, and it helps you notice connections between components that you might miss in a back-and-forth conversation.

Targeted questions also work well. "What does the `GameBoard` class do?" or "How does data flow from the API handler to the database?" are the kinds of questions that get useful answers. The AI can give you a quick summary that would take you several minutes to piece together on your own.



Verify what the AI tells you. AI can be wrong about unfamiliar codebases in the same ways you can. It might misread the purpose of a class, overlook an important side effect, or describe what the code *should* do rather than what it *actually* does. Use the AI's explanations as a starting point, then confirm them by reading the code yourself. If the AI says a method "returns the shortest path" but you see it's actually returning all paths, you

need to catch that before you build on top of it. Interviewers also want to see you in the code, not just in the chat. Someone who only reads AI summaries and never opens the files directly doesn't inspire confidence.

Also use the AI to explain patterns or syntax you don't recognize. If the codebase uses decorators, generics, or framework-specific idioms you haven't seen before, there's no shame in asking. The interviewer would rather see you use the AI to quickly understand unfamiliar syntax than watch you stare at it confused for two minutes.

Don't let anyone rush you

There's a natural pressure to start coding immediately. The clock is ticking, the interviewer is watching, and you feel like you should be producing something. Resist that instinct. Five to ten minutes spent reading the codebase will more than pay for itself during implementation, when your prompts reference actual class names and methods instead of vague descriptions. Interviewers expect it. Jumping straight to prompting without reading is one of the clearest failure signals they watch for.

The key is to narrate what you're learning as you go. "I see the `Board` uses a 2D grid of `Cell` objects, and each cell tracks its own state and neighbors" is exactly the kind of thing that makes the orientation phase feel productive to the interviewer rather than like dead air. It shows you're thinking critically about the code, and it gives the interviewer confidence that you'll make good decisions once you start building.

If you feel pressure to start coding, it's completely fine to say "I want to make sure I understand the codebase before I start making changes." No reasonable interviewer will penalize you for that.